

# IF • ELIF • ELSE

Conditional Statements: Understanding how Python makes decisions.

LEARN • BUILD • EXPLORE

## 01 Introduction to Decisions in Code

In the real world, we make decisions constantly: *"If it rains, I will take an umbrella; otherwise, I will wear sunglasses."* Programs need the exact same intelligence. By default, Python executes code line-by-line from top to bottom. **Conditional statements** allow a program to skip or execute specific blocks of code depending on whether a given condition is **True** or **False**.

### 💡 Core Concept: Boolean Evaluation

Every decision in Python boils down to a single question: **Is this expression True or False?** These two values are called Booleans.

## 02 Comparison Operators

To form a condition, we compare values using **Comparison (Relational) Operators**. They evaluate two values and return either **True** or **False**.

| Operator           | Meaning                  | Example                  | Result       |
|--------------------|--------------------------|--------------------------|--------------|
| <code>==</code>    | Equal to                 | <code>5 == 5</code>      | <b>True</b>  |
| <code>!=</code>    | Not equal to             | <code>10 != 5</code>     | <b>True</b>  |
| <code>&gt;</code>  | Greater than             | <code>7 &gt; 12</code>   | <b>False</b> |
| <code>&lt;</code>  | Less than                | <code>3 &lt; 8</code>    | <b>True</b>  |
| <code>&gt;=</code> | Greater than or equal to | <code>18 &gt;= 18</code> | <b>True</b>  |
| <code>&lt;=</code> | Less than or equal to    | <code>4 &lt;= 2</code>   | <b>False</b> |

```
# Live Comparison Example
x = 15
y = 20
print(x < y) # Output: True
print(x == y) # Output: False
```

-----  
TERMINAL OUTPUT

**True**  
**False**

## 04 The `else` Statement

What happens when the `if` condition evaluates to `False`? Use the `else` statement to specify an alternative block of code that runs when the primary condition fails.

```
# Syntax
if condition:
    statement_1
else:
    statement_2
```

```
# Example: Pass / Fail Check
marks = 42
```

```
if marks >= 50:
    print("Pass")
else:
    print("Keep practicing")
```

-----  
TERMINAL OUTPUT

Keep practicing

**Execution Breakdown:** Python checks `marks >= 50`. Since 42 is not greater than or equal to 50, the result is `False`. Python skips the `if` body and immediately executes the `else` block.

## 05 The `elif` Statement (Multiple Conditions)

When you have more than two possibilities, chaining multiple `if` statements can be inefficient. The `elif` (short for "else if") allows you to test several conditions sequentially.

```
# Example: Grade Evaluation
marks = 76
```

```
if marks >= 90:
    print("Excellent")
elif marks >= 75:
    print("Very Good")
elif marks >= 50:
    print("Pass")
else:
    print("Keep Practicing")
```

-----  
TERMINAL OUTPUT

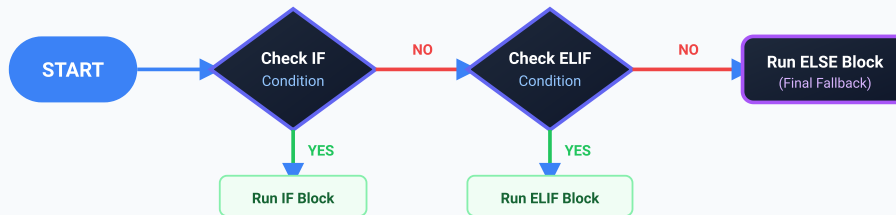
Very Good

### Execution Order (Top-to-Bottom)

1. Python tests `if marks >= 90` (76 >= 90 is `False`).
2. Python moves to first `elif`: `marks >= 75` (76 >= 75 is `True`).
3. It executes `print("Very Good")`.

## 07 Decision Flow Diagram

Visualizing how Python navigates conditional blocks:



## 08 Important Python Rules to Remember

- **Colon Required:** Always place a colon `:` after `if`, `elif`, and `else` headers.
- **Consistent Indentation:** Standard practice is 4 spaces. Mixing tabs and spaces throws an `IndentationError`.
- **Case Sensitivity:** Keywords (`if`, `elif`, `else`) and Booleans (`True`, `False`) must be exact. Writing `TRUE` or `If` will cause errors.
- **Operator Distinction ( `==` vs `=` ):**
  - `==` is for **Comparison** (checks equality).
  - `=` is for **Assignment** (stores a value in a variable).

```
# Comparison vs Assignment
x = 10 # Storing 10 in variable x (Assignment)
x == 10 # Evaluating if x equals 10 (Comparison -> Returns True)
```

## 09 Common Beginner Mistakes

### ✗ MISSING COLON

```
if age >= 18
    print("Eligible")
```

### ✓ CORRECTED

```
if age >= 18:
    print("Eligible")
```

### ✗ MISSING INDENTATION

```
if age == 18:
print("18")
```

### ✓ CORRECTED

```
if age == 18:
    print("18")
```

### ✗ USING '=' INSIDE AN IF CONDITION

```
if age = 18: # SyntaxError: invalid syntax
```

## 10 Practice Exercises

Write your own Python solution for each task. Practice before checking notes!

### Practice 1: Positive, Negative, or Zero

Basic

Write a program that takes a variable `num` and prints whether it is Positive, Negative, or Zero.

[ Write your code here ]

### Practice 2: Voting Eligibility

Basic

Check if `age >= 18`. Print "Eligible to Vote" or "Not Eligible".

[ Write your code here ]

### Practice 3: Even or Odd Number

Logic

Use the modulus operator ( `num % 2 == 0` ) to check if a number is Even or Odd.

[ Write your code here ]

## 11 Mini Challenge

### 🎯 Build a Simple Grade Calculator

Create a script using `if`, `elif`, and `else` that converts a student's score into a grade based on these criteria:

- 90 and above → Grade A
- 75 to 89 → Grade B
- 50 to 74 → Grade C
- Below 50 → Needs Improvement

### ⚡ Quick Recap

- `if`: Evaluates 1st condition.
- `elif`: Evaluates secondary conditions.
- `else`: Catches everything else.
- Conditions evaluate to `True` or `False`.

### NIL LABS

LEARN • BUILD • EXPLORE

"Small steps. Big goals."